1. Problem Title

Move Empty Chocolate Packets to the End

2. Problem Statement (Short Summary)

A chocolate factory packs chocolates into packets.

The packets are represented as an **array of integers**.

Some packets are **empty (value = 0)**.

Your task is to **move all empty packets (0s) to the end of the array while keeping the order of non-zero elements unchanged**.

3. Examples

Example 1

Input

8

4 5 0 1 9 0 5 0

Output

4 5 1 9 5 0 0 0

Explanation

Three empty packets (0) are moved to the end.

Example 2

Input

6

6 0 1 8 0 2

Output

6 1 8 2 0 0

4. Constraints

$1 \le N \le 1000$

$0 \le arr[i] \le 100$

5. Core Concept (Pattern)

Two Pointer Technique

This is the same pattern used in:

Move Zeroes (LeetCode 283)

Partition problems

Array compaction

6. Thought Process

Brute Force Idea

Create a new array.

1. Add all non-zero elements
2. Then add zeros

But this requires **extra space**.

Optimized Idea (Two Pointers)

Maintain pointer pos representing the **next position for non-zero element**.

Traverse the array:

If element ≠ 0

swap(arr[pos], arr[i])

pos++

This shifts all non-zero elements forward.

7. Visualization

Example

[4,5,0,1,9,0,5,0]

Step by step

pos=0

4 → swap → pos=1

5 → swap → pos=2

0 → skip

1 → swap → pos=3

9 → swap → pos=4

0 → skip

5 → swap → pos=5

0 → skip

Result

[4,5,1,9,5,0,0,0]

8. Pseudocode

```
pos = 0

for i in range(n):
```

```
    if arr[i] != 0:

        swap(arr[pos], arr[i])
        pos++
```

9. Code Implementation

Python

```
n = int(input())
arr = list(map(int,input().split()))

pos = 0

for i in range(n):

    if arr[i] != 0:
        arr[pos], arr[i] = arr[i], arr[pos]
        pos += 1

print(*arr)
```

Java

```
import java.util.*;

class Main {

    public static void moveZeros(int[] arr){

        int pos = 0;

        for(int i = 0; i < arr.length; i++){

            if(arr[i] != 0){

                int temp = arr[pos];
                arr[pos] = arr[i];
                arr[i] = temp;
```

```
            pos++;
        }
    }
  }

  public static void main(String[] args){

    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();

    int[] arr = new int[n];

    for(int i=0;i<n;i++)
        arr[i] = sc.nextInt();

    moveZeros(arr);

    for(int x : arr)
        System.out.print(x + " ");
  }
}
```

10. Complexity

Time Complexity

O(n)

Space Complexity

O(1)

11. Common Mistakes

1. Using extra array unnecessarily
2. Not preserving order
3. Incorrect swap logic
4. Using nested loops

12. Dry Run

Input

[6,0,1,8,0,2]

i	arr[i]	pos	array
0	6	1	6 0 1 8 0 2
1	0	1	6 0 1 8 0 2
2	1	2	6 1 0 8 0 2
3	8	3	6 1 8 0 0 2
4	0	3	skip
5	2	4	6 1 8 2 0 0

13. Real Use Case

Similar patterns appear in:

- Data compression
- Memory compaction
- Filtering invalid records
- Log cleaning

14. Related Problems

LC 283 Move Zeroes

LC 27 Remove Element

LC 75 Sort Colors